

Exercício 1: Como funciona o princípio básico de ordenação do algoritmo *Bubble Sort*?

Exercício 2: Quais são as principais vantagens e desvantagens do *Bubble Sort*?

Exercício 3: Se tivermos um vetor inicial [5, 4, 3, 2, 1], qual será o primeiro par de elementos analisado pelo *Bubble Sort* na primeira varredura e o que acontecerá com eles?

Exercício 4: No código em linguagem C para o *Bubble Sort*, qual é o papel da variável `temp` dentro do bloco condicional?

```
c
```

```
if (arr[j] > arr[j + 1]) {  
    temp = arr[j];  
    arr[j] = arr[j + 1];  
    arr[j + 1] = temp;  
}
```

Exercício 5: Analisando a função `main` apresentada no arquivo, como é calculado dinamicamente o número total de elementos (tamanho) de um vetor de inteiros em C?

Exercício 6: Qual analogia do mundo real é frequentemente utilizada para explicar o comportamento do algoritmo *Insertion Sort*?

Exercício 7: Quais são as vantagens e desvantagens práticas do uso do algoritmo *Insertion Sort*?

Exercício 8: Considere o vetor {1, 3, 4, 2, 7, 9, 5, 6, 8}. No contexto do *Insertion Sort*, o que acontece imediatamente após o algoritmo identificar que o elemento 2 (no índice 3) está fora de ordem?

Exercício 9: No algoritmo *Insertion Sort*, a partir de qual índice do vetor o laço de repetição principal deve começar a iterar e por quê?

Exercício 10: Observe o trecho de código do *Insertion Sort*:

c

```
while (j >= 0 && arr[j] > key) {  
    arr[j + 1] = arr[j];  
    j = j - 1;  
}
```

Quais são as duas condições necessárias para que este laço while continue movendo elementos para a direita?